

● Deployed & verified — 2026-09-20

An AI resilience control tower for *manufacturing* supply chains.

SupplyChain Nexus turns a single supplier disruption event into a quantified impact, a compared set of recovery strategies, and an evidence-backed action plan — grounded end-to-end on Microsoft Foundry.
Course activity: From prototype to production with Microsoft Foundry

- Open live app
- Read full submission
- Repository README
- SupplyChain Nexus — Final Submission Pitch

5

Foundry agents live in production

5

Independent disruption scenarios

gpt-5.1

Current specialist model deployment

\$0

Invented numbers — every figure is tool-grounded

THE BUSINESS PROBLEM

A supplier delay is reported as one line. Its consequences cross the whole factory floor.

-
- 01 **Hours lost tracing manually** — planners hand-walk bills of material and inventory coverage instead of getting an instant answer.
 - 02 **Expensive knee-jerk responses** — teams commit to the first mitigation option without comparing cost, delay, and risk trade-offs.
 - 03 **Missed tier-1 commitments** — customer SLA and regulatory obligations get discovered after the fact, not before the decision.
 - 04 **Unauthorized spend** — financially material actions execute without the correct approval role in the loop.
 - 05 **No audit trail** — the reasoning behind the final call disappears the moment the meeting ends.
-

INTENDED USERS

Five roles. One shared source of truth.

For the modeled SUP-042 event, the system traces affected materials through the BOM to products, plants, and customer orders — then serves the same evidence to five different jobs.

ROLE
NEED
EXPERIENCE
Supply-chain manager Decide whether and how to intervene Control tower, impact summary, simulation comparison, approval gate
MRP / production planner Understand material runway and plant constraints Digital twin, inventory coverage, affected orders, scenario controls
Procurement lead Select and authorize alternate sourcing or expedite actions Recovery plan, supplier & contract evidence, action list
Executive operations leader Review financial exposure and business trade-offs Executive summary, revenue protected, cost, delay, SLA risk
AI / platform engineer Diagnose, evaluate, and improve the system Agent traces, tool telemetry, evaluation results, audit records

One event in. A grounded, approvable recovery plan out.

SupplyChain Nexus is a multi-agent resilience workflow: deterministic tools compute every number, four Foundry specialist agents interpret those numbers for a human decision-maker, and an approval gate sits outside the model's authority entirely.



Grounded, not guessed

Every dollar figure, order count, and coverage day comes from a deterministic engine. Agents interpret facts; they never compute them.



5 live scenarios

The same pipeline runs correctly across five independently modeled suppliers, severities, and durations — not a single cherry-picked demo path.



Caught our own mistake

An early evaluation scored 0–40% because it was graded against a math benchmark, not this domain. We found it, rebuilt the evaluator, and re-ran it clean.

Why *four agents* beat one generalist.

A single agent would have to reason about graph traversal, numerical simulation, contracts, procurement policy, and authorization inside one context window. That is exactly where ungrounded numbers and unclear ownership come from.

ONE GENERALIST AGENT

Everything, one context

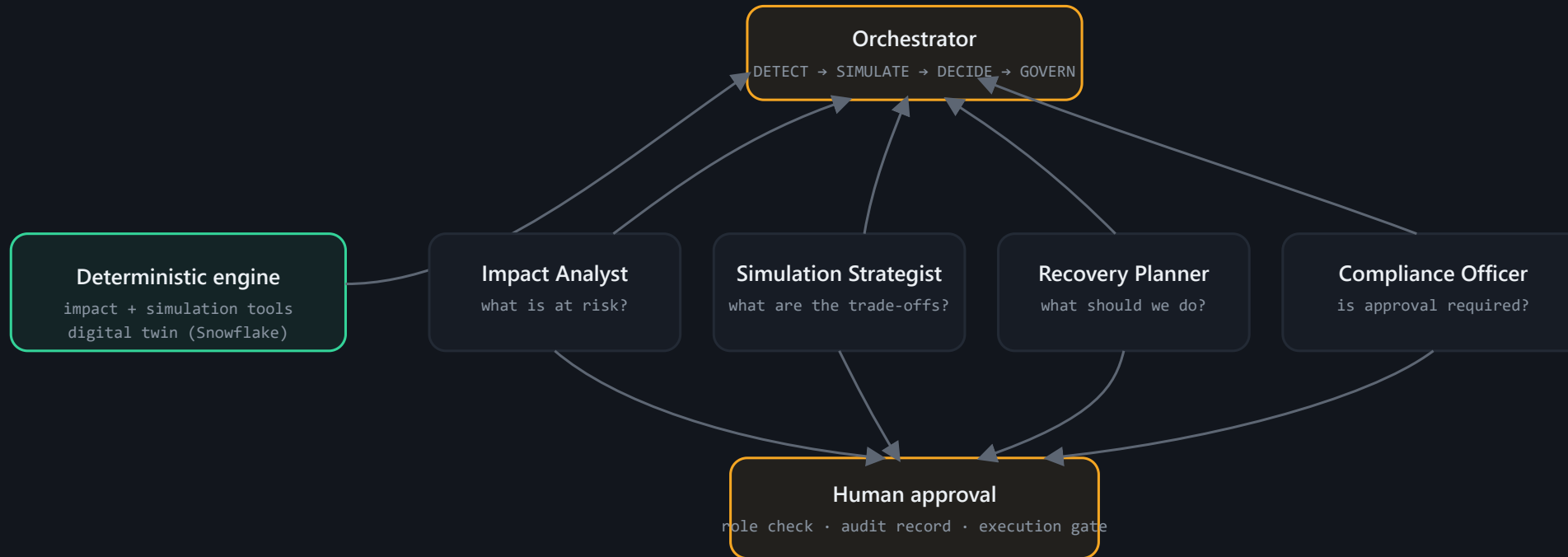
- Numbers get calculated from memory, not tools — harder to catch a wrong figure
- One oversized prompt mixes impact math, policy, and approval logic
- No clear owner for a bad answer — is it the data, the reasoning, or the policy?
- Hard to evaluate: one pass/fail score hides which responsibility actually failed
- The model can reason its way past an approval boundary it shouldn't cross

FOUR SPECIALISTS + ONE GATE

Narrow objective, own tools

- **Separation of concerns** — each agent has one question and one tool set
- **Grounding** — decision-critical figures come back from deterministic tools, not the model
- **Testability** — each specialist is evaluated independently against its own facts
- **Explainability** — every stage emits its own input, output, and trace
- **Governance** — the Recovery Planner can recommend; only the approval service can execute

Four specialists. One deterministic engine. One human gate.



1. **Event received** — the orchestrator validates the disruption event (supplier, duration) and opens a correlation trace.
2. **Impact Analyst** calls the deterministic engine for revenue at risk, affected plants/materials, and inventory coverage.
3. **Simulation Strategist** receives that impact package and compares five recovery strategies on cost, delay, and SLA risk.
4. **Recovery Planner** receives the simulation results and justifies the recommended strategy with concrete next actions.
5. **Compliance Officer** receives the recommended strategy and its cost, then states the required approval role.
6. **Human approval** reviews the full evidence trail and approves or rejects — outside the model's authority entirely.

Each agent answers exactly one question.

"What is at risk?"

Impact Analyst

Leads with revenue at risk, orders, and inventory runway. Names constrained materials and plants.

`calculate_supplier_impact`

data: digital twin – supplier, BOM, plant & order records (Snowflake)

"What are the trade-offs?"

Simulation Strategist

Compares five recovery strategies on cost, delay, and SLA risk. Never picks a winner.

`simulate_strategies`

data: deterministic scenario engine – freight, capacity & cost assumptions

"What should we do?"

Recovery Planner

Justifies the recommended strategy on net benefit and lists the concrete actions required.

`recommend_recovery`

data: recovery policy rules, procurement guidelines, supplier contracts

"Is approval required?"

Compliance Officer

States who must approve and what must be audited, gated on a hard cost threshold.

`$250,000 threshold`

data: approval policy – role matrix & financial threshold

Every specialist carries the same instruction: **"You never calculate, estimate, or invent numbers. Every figure you mention must appear verbatim in the VERIFIED FACTS supplied to you."** Grounding is enforced in the prompt, then verified in evaluation.

LIVE • NOT A MOCK

Two real disruptions, computed by the deterministic engine against live data.

HIGH severity • zero inventory buffer	
SUP-078 • 10-day disruption	
Caldera Precision GmbH • 15 plants affected	
Revenue at risk	\$1,624,410,489
Customer orders at risk	1,350
Inventory coverage	0.0 days
Recommended strategy	ALTERNATE_SUPPLIER
Net benefit	\$1,543,004,965

CRITICAL severity • 8.2 days buffer	
SUP-042 • 14-day disruption	
Kaizen Precision Components GmbH • 3 plants affected	
Revenue at risk	\$16,700,100
Customer orders at risk	127
Inventory coverage	8.2 days
Recommended strategy	ALTERNATE_SUPPLIER
Net benefit	\$15,680,095

The bug we found was in our own test, not the agents.

An early Foundry evaluation run scored 0–40% across all five agents. The honest instinct was to assume the agents were broken.

Root cause: the evaluation was configured against the AIME 2025 math benchmark — a dataset with zero relation to supply-chain reasoning. The fix wasn't to accept the score, or hand-pick a flattering benchmark. We built `local_agent_eval.py`: a rubric evaluator that judges each specialist against the exact `VERIFIED FACTS` it received, checking rubric adherence *and* fabricated numbers. Re-run against real disruption scenarios, every agent passes consistently.

Relevance & groundedness

Numeric claims are checked verbatim against the facts block, not vibes.

Task adherence

Impact → comparison → recommendation → approval, in order, every time.

Policy adherence

Approval threshold and role logic hold even at \$1.6B revenue exposure.

Every step leaves a trace.

The Agent Trace view renders the real tool calls the orchestrator makes — not a scripted animation.

Correlation ID

Every request threads one ID from browser to model to tool.

Tool telemetry

Name, arguments, latency, and result status per call.

Model metadata

Deployment, version, token counts, and duration per specialist.

Audit record

Approval role, decision, and policy version tied to the case.

Prototype today. Production is four phases away.

PHASE 1 • NOW

Prototype

- Synthetic disruption scenarios
 - Agent-to-tool loop validated
 - Traces & audit captured
-

PHASE 2

Controlled pilot

- Read-only ERP / MRP connectors
 - Entra ID, RBAC, environments
 - Evaluation dataset + release gate
-

PHASE 3

Production

- Durable case & audit storage
 - Approved PO / logistics actions
 - Continuous eval + canary rollout
-

PHASE 4

Optimization

- Learn from accepted/rejected plans
- Cost/quality model tuning
- Proactive risk detection

This isn't a slide. It's a running system.

Live endpoint

`supplychain-nexus-app.azurewebsites.net`
`/api/health · /api/scenarios · /api/agents/run`

Foundry project

`supplychain-nexus-dev`
5 agents, all versioned, all on `gpt-5.1`

13 / 13

Grounded agents. Real disruptions. A human still decides.

SupplyChain Nexus — built for the "From prototype to production with Microsoft Foundry" course activity.

[Open live app](#)

[Read full submission](#)

[Repository README](#)

[SupplyChain Nexus — Final Submission Pitch](#)